

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2-DEBUGGING & OPTIMISATION TASK

Course Code:	CSA1205	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL2-DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:		Reg.No.:	
Department:		Date:	

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.
2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.
3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.
4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.
5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors.

Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

I V. Reddy Sreya (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Identification of Errors / Bugs(20%)	Debugging Approach & Analysis(20%)	Correctness of Debugged Solution(25%)	Optimization Techniques Applied(20%)	Testing and Documentation(15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

1. Signed Addition Using 2's Complement

For 8-bit signed integers:

- +120 = 01111000
- +50 = 00110010

Addition:

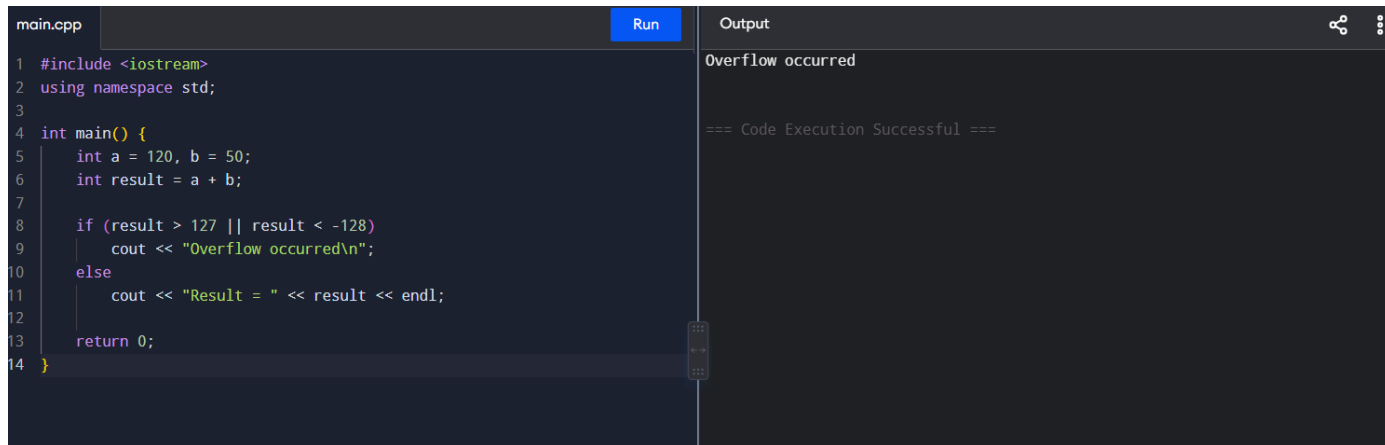
```
01111000 (+120)
+ 00110010 (+50)
-----
10101010
```

10101010 is negative in 2's complement, but the expected result is 170, which is outside the 8-bit signed range:

Range = -128 to +127

Therefore, overflow has occurred.

Rectified Code and Output:



```
main.cpp Run Output
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int a = 120, b = 50;
6     int result = a + b;
7
8     if (result > 127 || result < -128)
9         cout << "Overflow occurred\n";
10    else
11        cout << "Result = " << result << endl;
12
13    return 0;
14 }
```

Output

Overflow occurred

=== Code Execution Successful ===

Correct solution: Check whether the result exceeds the signed range before storing it in 8 bits.

2. Ripple Carry Adder → Carry Look-Ahead Adder

Problem

In a Ripple Carry Adder (RCA), each carry must wait for the previous carry:

$C_0 \rightarrow FA_0 \rightarrow C_1 \rightarrow FA_1 \rightarrow C_2 \rightarrow FA_2 \rightarrow C_3 \rightarrow \dots$

For an 8-bit adder, the carry may propagate through all 8 bits, causing large delay.

Optimized solution: Carry Look-Ahead Adder

CLA calculates carries in parallel using:

$P_i = A_i \text{ XOR } B_i$

$G_i = A_i \text{ AND } B_i$

For example:

$C_1 = G_0 + P_0C_0$

$C_2 = G_1 + P_1G_0 + P_1P_0C_0$

Thus, carries don't have to wait for every previous full adder.

Code and Output:

main.cpp	Run	Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 int main() { 5 int A = 25; 6 int B = 35; 7 int C = 0; 8 9 int P[8], G[8], carry[9]; 10 carry[0] = C; 11 12 for (int i = 0; i < 8; i++) { 13 int a = (A >> i) & 1; 14 int b = (B >> i) & 1; 15 16 P[i] = a ^ b; 17 G[i] = a & b; 18 } 19 20 for (int i = 0; i < 8; i++) 21 carry[i + 1] = G[i] (P[i] & carry[i]); 22 23 int result = 0; 24 25 for (int i = 0; i < 8; i++) 26 result = (P[i] ^ carry[i]) << i; 27 28 cout << "Result = " << result << endl; 29 30 return 0; 31 }</pre>		<pre>Result = 60 --- Code Execution Successful ---</pre>

Optimization: CLA reduces carry propagation delay compared with RCA.

3. Booth's Algorithm – Signed Multiplication

The common errors are:

1. Checking the wrong bit pair.
2. Using a logical shift instead of an arithmetic shift.
3. Not sign-extending negative numbers.
4. Updating Q-1 incorrectly.

Booth's bit-pair rules

Check:

Q0 Q-1

Q0 Q-1 Operation

00 No operation

01 $A = A + M$

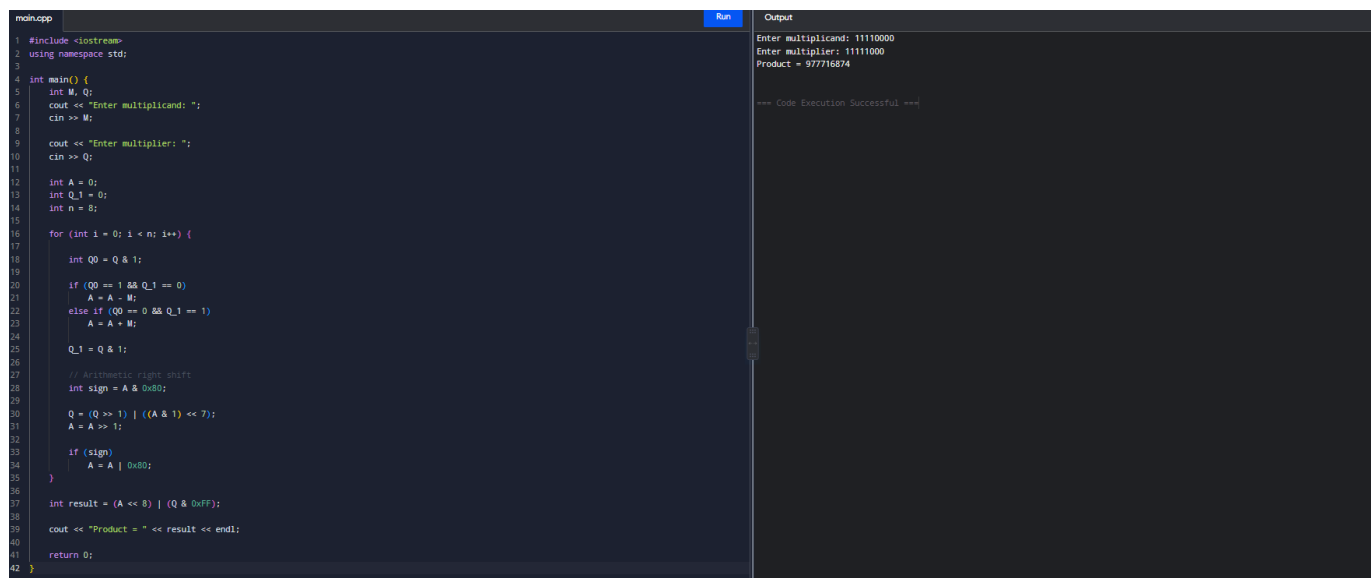
10 $A = A - M$

11 No operation

After every operation, perform an Arithmetic Right Shift on:

A, Q, Q-1

Rectified Code and Output:



```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int M, Q;
6     cout << "Enter multiplicand: ";
7     cin >> M;
8
9     cout << "Enter multiplier: ";
10    cin >> Q;
11
12    int A = 0;
13    int Q_1 = 0;
14    int n = 8;
15
16    for (int i = 0; i < n; i++) {
17        int Q0 = Q & 1;
18
19        if (Q0 == 1 && Q_1 == 0)
20            A = A - M;
21        else if (Q0 == 0 && Q_1 == 1)
22            A = A + M;
23
24        Q_1 = Q & 1;
25
26        // Arithmetic right shift
27        int sign = A & 0x80;
28
29        Q = (Q >> 1) | ((A & 1) << 7);
30        A = A >> 1;
31
32        if (sign)
33            A = A | 0x80;
34    }
35
36    int result = (A << 8) | (Q & 0xFF);
37
38    cout << "Product = " << result << endl;
39
40    return 0;
41 }
```

Output

```
Enter multiplicand: 11110000
Enter multiplier: 11110000
Product = 977716874

=== Code Execution Successful ===
```

Main correction

For negative operands, sign extension must be preserved during the arithmetic right shift.

For example:

11110000

Arithmetic right shift:

11111000

not:

01111000

4. Restoring Division → Non-Restoring Division

Problem with restoring division

In restoring division:

1. Subtract divisor.
2. Check result.
3. If negative, restore the previous value by adding divisor again.

So there are repeated operations:

$$A = A - M$$

If $A < 0$:

$$A = A + M \quad \leftarrow \text{unnecessary restoration}$$

This increases execution time.

Non-restoring division

Non-restoring division avoids immediate restoration.

If the partial remainder is positive:

$$A = A - M$$

If it is negative:

$$A = A + M$$

The next operation automatically corrects the remainder.

Simple code

```
#include <iostream>
using namespace std;
```

```

int main() {
    int dividend, divisor;

    cout << "Enter dividend: ";
    cin >> dividend;

    cout << "Enter divisor: ";
    cin >> divisor;

    if (divisor == 0) {
        cout << "Division by zero is not allowed";
        return 0;
    }

    int A = 0;
    int Q = dividend;
    int M = divisor;
    int n = 8;

    for (int i = 0; i < n; i++) {

        A = (A << 1) | ((Q >> 7) & 1);
        Q = Q << 1;

        if (A >= 0)
            A = A - M;
        else
            A = A + M;

        if (A >= 0)
            Q = Q | 1;
    }

    if (A < 0)
        A = A + M;

    cout << "Quotient = " << Q << endl;
    cout << "Remainder = " << A << endl;

    return 0;
}

```

Advantage: Non-restoring division eliminates the repeated restore operation, so it is generally faster.

5. IEEE 754 Single Precision – Small + Large Number

Single precision uses:

1 sign bit
8 exponent bits
23 mantissa bits

Problem

Suppose:

$$1.0 \times 10^{20} + 1.0$$

Before addition, exponents must be aligned:

$$1.0 \times 10^{20}$$
$$0.000... \times 10^{20}$$

The very small number may lose its significant bits during alignment because only **23 fraction bits** are available.

This causes **rounding error**.

Rectified C++ code

Use double instead of float when higher precision is required.

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    float a = 1.0e20f;
    float b = 1.0f;

    float result1 = a + b;

    double x = 1.0e20;
    double y = 1.0;

    double result2 = x + y;

    cout << setprecision(17);
```



```
cout << "Using float : " << result1 << endl;  
cout << "Using double : " << result2 << endl;  
  
return 0;  
}
```

Ways to improve accuracy

- Use double instead of float.
- Use appropriate normalization.
- Use guard/round/sticky bits in hardware.
- Use **Kahan summation** when adding many floating-point values.
- Avoid adding numbers with extremely different magnitudes when possible.
- Use higher precision for scientific calculations.